

## **LAB- 4**

**Course: Compiler Design Laboratory**

**SLOT: L23+L24**

**Name: Shrri Dharshan D R**

**Reg No: 23BPS1090**

### **OBJECTIVE**

1. To understand recursive descent parsing for LL(1) grammars.
2. To implement a recursive descent parser for arithmetic expressions.
3. To design top-down parsers using Lex and Yacc.
4. To validate arithmetic expressions, assignment statements, and Boolean expressions.
5. To ensure correct operator precedence, associativity, and parentheses matching.
6. To identify lexical and syntax errors effectively.

### **PROBLEM STATEMENT**

Design and implement syntax analyzers for different language constructs using recursive descent parsing and Lex-Yacc. The analyzers must recognize and validate arithmetic expressions, assignment statements, and Boolean expressions based on the provided grammars. The programs should tokenize the input, apply grammar rules, detect syntax and lexical errors, and show appropriate success or error messages for valid and invalid inputs.

### **AIM**

To design and implement syntax analyzers using recursive descent parsing and Lex-Yacc techniques for checking arithmetic expressions, assignment statements, and Boolean expressions based on specified context-free grammars.

### **PROCEDURE**

1. Study the given grammars for arithmetic, assignment, and Boolean expressions.
2. Implement a recursive descent parser in C for the provided LL(1) grammar.
3. Define functions for each non-terminal in the grammar.
4. Read the input expression and parse it with recursive function calls.
5. Set error flags when you encounter invalid syntax.
6. Develop Lex programs to break down identifiers, numbers, operators, and keywords.
7. Write Yacc programs to outline grammar rules and operator precedence.
8. Combine Lex and Yacc to carry out top-down parsing.
9. Compile and run the programs using Lex, Yacc, and GCC commands.
10. Test the parser with valid and invalid inputs, and note the output.

## Write a recursive descent parser for the CFG language

### lab Question 1:

We will build a parser for:

$$E \rightarrow T E'$$
$$E' \rightarrow + T E' \mid \varepsilon$$
$$T \rightarrow F T'$$
$$T' \rightarrow * F T' \mid \varepsilon$$
$$F \rightarrow (E) \mid \text{id}$$

This grammar handles:

- identifiers (id)
- parentheses
  - operator
- - operator
- proper precedence and associativity

It is **LL(1)** and suitable for recursive descent.

### Example Run

#### Input:

a + b \* c

#### Output:

Parsing Successful!

### Example of Invalid Input:

a \* + b

#### Output:

Syntax Error in F

### CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
char input[50];
int i = 0, error = 0;
```

```
void E();
void Eprime();
void T();
```

```
void Tprime();  
void F();
```

```
void E() {  
    T();  
    Eprime();  
}
```

```
void Eprime() {  
    if (input[i] == '+') {  
        i++;  
        T();  
        Eprime();  
    }  
}
```

```
void T() {  
    F();  
    Tprime();  
}
```

```
void Tprime() {  
    if (input[i] == '*') {  
        i++;  
        F();  
        Tprime();  
    }  
}
```

```
void F() {  
    if (input[i] == '(') {  
        i++;  
        E();  
        if (input[i] == ')')  
            i++;  
        else  
            error = 1;  
    }  
    else if (input[i] >= 'a' && input[i] <= 'z') {  
        i++;  
    }  
    else {  
        error = 1;  
    }  
}
```

```

    }
}

int main() {
    printf("Enter the expression: ");
    scanf("%s", input);

    E();

    if (strlen(input) == i && error == 0)
        printf("Parsing Successful!\n");
    else
        printf("Syntax Error in F\n");

    return 0;
}

```

## OUTPUT:

### PARSING SUCCESSFUL

```

shrridharshan@SD:~$ nano rd_parser.c
shrridharshan@SD:~$ gcc rd_parser.c -o rd
shrridharshan@SD:~$ ./rd
Enter the expression: a+b*c
Parsing Successful!

```

### SYNTAX ERROR IN F

```

shrridharshan@SD:~$ nano rd_parser.c
shrridharshan@SD:~$ gcc rd_parser.c -o rd
shrridharshan@SD:~$ ./rd
Enter the expression: a*+b
Syntax Error in F

```

## Lab Question 2

You are developing a module for a scientific calculator that must validate basic arithmetic expressions before evaluation.

### Task:

Write LEX and YACC programs to implement a **top-down parser** that recognizes arithmetic expressions involving:

- integers
- operators: +, -, \*, /
- parentheses ( )

### Grammar to Use:

$$E \rightarrow E + T \mid E - T \mid T$$
$$T \rightarrow T * F \mid T / F \mid F$$
$$F \rightarrow (E) \mid \text{NUM}$$

Outputs Required:

1. Display:
  - “Valid Expression” if the input matches the grammar
  - “Invalid Expression” otherwise
2. Show the parsing steps (shift/reduce actions or rule applications).
3. Handle lexical errors using LEX rules.

### CODE:

#### LEX

```
%{  
  
#include "y.tab.h"  
  
#include <stdio.h>  
  
%}  
  
%%  
  
[0-9]+    { printf("TOKEN: NUM (%s)\n", yytext); return NUM; }  
"+"      { printf("TOKEN: +\n"); return PLUS; }
```

```

"-"      { printf("TOKEN: -\n"); return MINUS; }
"*"      { printf("TOKEN: *\n"); return MUL; }
"/"      { printf("TOKEN: /\n"); return DIV; }
"("      { printf("TOKEN: (\n"); return LPAREN; }
")"      { printf("TOKEN: )\n"); return RPAREN; }
[ \t\n]+ ;
.        { printf("Lexical Error: %s\n", yytext); return INVALID; }

```

%%

## YACC

```
%{
```

```
#include <stdio.h>
```

```
int yylex();
```

```
int yyerror(const char *s);
```

```
%}
```

```
%token NUM
```

```
%token PLUS MINUS MUL DIV
```

```
%token LPAREN RPAREN
```

```
%token INVALID
```

```
%left PLUS MINUS
```

```
%left MUL DIV
```

%%

```
E : E PLUS T { printf("Reduce: E → E + T\n"); }
```

```
    | E MINUS T { printf("Reduce: E → E - T\n"); }
```

```
    | T        { printf("Reduce: E → T\n"); }
```

```
;
```

```

T : T MUL F { printf("Reduce: T → T * F\n"); }
    | T DIV F { printf("Reduce: T → T / F\n"); }
    | F      { printf("Reduce: T → F\n"); }
    ;

F : LPAREN E RPAREN { printf("Reduce: F → (E)\n"); }
    | NUM            { printf("Reduce: F → NUM\n"); }
    ;
%%

int main() {
    printf("Enter expression:\n");
    if (yyparse() == 0)
        printf("\nValid Expression\n");
    else
        printf("\nInvalid Expression\n");
    return 0;
}

int yyerror(const char *s) {
    printf("\nInvalid Expression\n");
    return 0;
}

```

## OUTPUT:

### VALID EXPRESSION:

```
shrridharshan@SD:~$ nano expr.l
shrridharshan@SD:~$ nano expr.y
shrridharshan@SD:~$ yacc -d expr.y
shrridharshan@SD:~$ lex expr.l
shrridharshan@SD:~$ gcc lex.yy.c y.tab.c -o expr -ll
shrridharshan@SD:~$ ./expr
Enter expression:
10+20*3
TOKEN: NUM (10)
Reduce: F → NUM
Reduce: T → F
TOKEN: +
Reduce: E → T
TOKEN: NUM (20)
Reduce: F → NUM
Reduce: T → F
TOKEN: *
TOKEN: NUM (3)
Reduce: F → NUM
Reduce: T → T * F
Reduce: E → E + T

Valid Expression
shrridharshan@SD:~$ |
```

### INVALID EXPRESSION:

```
shrridharshan@SD:~$ nano expr.l
shrridharshan@SD:~$ nano expr.y
shrridharshan@SD:~$ yacc -d expr.y
shrridharshan@SD:~$ lex expr.l
shrridharshan@SD:~$ gcc lex.yy.c y.tab.c -o expr -ll
^[[Ashrridharshan@SD:~$ ./expr
Enter expression:
10+*3
TOKEN: NUM (10)
Reduce: F → NUM
Reduce: T → F
TOKEN: +
Reduce: E → T
TOKEN: *

Invalid Expression

Invalid Expression
```

### Lab Question 3

A compiler front-end needs to recognize assignment statements during syntax analysis.

#### Task:

Write LEX and YACC programs to implement a **recursive-descent-style top-down parser** that recognizes assignment statements of the form:

identifier = expression;

#### Rules to Support:

- ID = ID + ID;
- ID = ID \* ID;
- ID = NUM;
- Allow nested parentheses in expressions

#### Grammar Provided:

$S \rightarrow ID = E ;$

$E \rightarrow E + T \mid E * T \mid T$

$T \rightarrow ID \mid NUM \mid (E)$

#### Outputs Required:

1. Print token stream from LEX
2. Print "Valid assignment statement" or error message
3. Generate a parse tree or abstract syntax tree (optional bonus)

#### CODE:

##### LEX

```
%{  
  
#include "y.tab.h"  
  
#include <stdio.h>  
  
%}  
  
%%  
  
[a-zA-Z]+ { printf("TOKEN: ID (%s)\n", yytext); yylval.ch = yytext[0]; return ID; }  
[0-9]+ { printf("TOKEN: NUM (%s)\n", yytext); yylval.ch = yytext[0]; return NUM; }
```

```

"="      { printf("TOKEN: =\n"); return ASSIGN; }
"+"      { printf("TOKEN: +\n"); return PLUS; }
"*"      { printf("TOKEN: *\n"); return MUL; }
";"      { printf("TOKEN: ;\n"); return SEMI; }
"("      { printf("TOKEN: (\n"); return LPAREN; }
")"      { printf("TOKEN: )\n"); return RPAREN; }
[ \t\n]+ ;
.        { printf("Lexical Error: %s\n", yytext); return INVALID; }
%%

```

## YACC

```

%{
#include <stdio.h>
#include <stdlib.h>

struct node {
    char data;
    struct node *left;
    struct node *right;
};

struct node* createNode(char data, struct node* left, struct node* right);
void printExpressionTree(struct node* root);

int yylex();
int yyerror(const char *s);

struct node* root;
%}

```

```
%union {
```

```
    char ch;
```

```
    struct node* node;
```

```
}
```

```
%token <ch> ID NUM
```

```
%token ASSIGN PLUS MUL SEMI
```

```
%token LPAREN RPAREN
```

```
%token INVALID
```

```
%type <node> S E T
```

```
%left PLUS
```

```
%left MUL
```

```
%%
```

```
S : ID ASSIGN E SEMI
```

```
{
```

```
    root = createNode('=', createNode($1, NULL, NULL), $3);
```

```
    printf("\nValid assignment statement\n");
```

```
}
```

```
;
```

```
E : E PLUS T { $$ = createNode('+', $1, $3); }
```

```
| E MUL T { $$ = createNode('*', $1, $3); }
```

```
| T { $$ = $1; }
```

```
;
```

```
T : ID { $$ = createNode($1, NULL, NULL); }
```

```

| NUM          { $$ = createNode($1, NULL, NULL); }
| LPAREN E RPAREN { $$ = $2; }
;
%%

struct node* createNode(char data, struct node* left, struct node* right) {
    struct node* n = (struct node*)malloc(sizeof(struct node));
    n->data = data;
    n->left = left;
    n->right = right;
    return n;
}

void printExpressionTree(struct node* root) {
    if (!root) return;

    printf("    %c\n", root->data);
    printf("    /\n");
    printf("    %c  %c\n", root->left->data, root->right->data);

    if (root->right && (root->right->left || root->right->right)) {
        printf("        /\n");
        printf("        %c  %c\n",
            root->right->left->data,
            root->right->right->data);

        if (root->right->right &&
            root->right->right->left &&
            root->right->right->right) {
            printf("            /\n");
            printf("            %c  %c\n",

```

```
        root->right->right->left->data,  
        root->right->right->right->data);  
    }  
}  
}  
  
int main() {  
    printf("Enter assignment statement:\n");  
    if (yyparse() == 0) {  
        printf("\nParse Tree:\n");  
        printExpressionTree(root);  
        printf("\n");  
    }  
    return 0;  
}  
  
int yyerror(const char *s) {  
    printf("\nInvalid assignment statement\n");  
    return 0;  
}
```

## OUTPUT:

### VALID ASSIGNMENT STATEMENT

```
shrridharshan@SD:~$ nano assign.l
shrridharshan@SD:~$ nano assign.y
shrridharshan@SD:~$ yacc -d assign.y
shrridharshan@SD:~$ lex assign.l
shrridharshan@SD:~$ gcc lex.yy.c y.tab.c -o assign -ll
shrridharshan@SD:~$ ./assign
Enter assignment statement:
a=b+c*d;
TOKEN: ID (a)
TOKEN: =
TOKEN: ID (b)
TOKEN: +
TOKEN: ID (c)
TOKEN: *
TOKEN: ID (d)
TOKEN: ;

Valid assignment statement

Parse Tree:
      =
     /\
    a  *
     /\
    +  d
```

### INVALID ASSIGNMENT STATEMENT

```
shrridharshan@SD:~$ nano assign.l
shrridharshan@SD:~$ nano assign.y
shrridharshan@SD:~$ yacc -d assign.y
shrridharshan@SD:~$ lex assign.l
shrridharshan@SD:~$ gcc lex.yy.c y.tab.c -o assign -ll
shrridharshan@SD:~$ ./assign
Enter assignment statement:
a = + b;
TOKEN: ID (a)
TOKEN: =
TOKEN: +

Invalid assignment statement
```

## Lab Question 4

You are creating an interpreter for a simple rule-based system that accepts Boolean expressions such as:

(a AND b) OR (c AND NOT d)

### Task:

Write LEX and YACC programs to implement a **top-down parser** for Boolean expressions including:

- Boolean identifiers
- Operators: AND, OR, NOT
- Parentheses

### Grammar Provided:

$B \rightarrow B \text{ OR } T \mid T$

$T \rightarrow T \text{ AND } F \mid F$

$F \rightarrow \text{NOT } F \mid \text{ID} \mid (B)$

### Expectations:

1. Read the expression from input
2. Tokenize Boolean keywords using LEX
3. Parser should validate structure and print success or error
4. Provide meaningful syntax error messages

### CODE:

#### LEX

```
%{  
  
#include "y.tab.h"  
  
#include <stdio.h>  
  
%}  
  
  
%%  
  
"AND" { printf("TOKEN: AND\n"); return AND; }  
  
"OR" { printf("TOKEN: OR\n"); return OR; }
```

```

"NOT"  { printf("TOKEN: NOT\n"); return NOT; }
[a-zA-Z]+ { printf("TOKEN: ID (%s)\n", yytext); return ID; }
"("    { printf("TOKEN: (\n"); return LPAREN; }
")"    { printf("TOKEN: )\n"); return RPAREN; }
[ \t\n]+ ;
.      { printf("Lexical Error: %s\n", yytext); return INVALID; }
%%

```

## YACC

```

%{
#include <stdio.h>

int yylex();
int yyerror(const char *s);
%}

```

```

/* Tokens */

%token ID AND OR NOT
%token LPAREN RPAREN
%token INVALID

```

```

/* Precedence */

%left OR
%left AND
%right NOT

```

```

%%

B : B OR T
  | T
  ;

```

T : T AND F

| F

;

F : NOT F

| ID

| LPAREN B RPAREN

;

%%

```
int main() {
```

```
    printf("Enter Boolean expression:\n");
```

```
    if (yyparse() == 0)
```

```
        printf("\nValid Boolean Expression\n");
```

```
    return 0;
```

```
}
```

```
int yyerror(const char *s) {
```

```
    printf("\nSyntax Error in Boolean Expression\n");
```

```
    return 0;
```

```
}
```

## OUTPUT:

### VALID BOOLEAN EXPRESSION

```
shrridharshan@SD:~$ nano bool.l
shrridharshan@SD:~$ nano bool.y
shrridharshan@SD:~$ yacc -d bool.y
shrridharshan@SD:~$ lex bool.l
shrridharshan@SD:~$ gcc lex.yy.c y.tab.c -o bool -ll
shrridharshan@SD:~$ ./bool
Enter Boolean expression:
(a AND b) OR (c AND NOT d)
TOKEN: (
TOKEN: ID (a)
TOKEN: AND
TOKEN: ID (b)
TOKEN: )
TOKEN: OR
TOKEN: (
TOKEN: ID (c)
TOKEN: AND
TOKEN: NOT
TOKEN: ID (d)
TOKEN: )
Valid Boolean Expression
```

### INVALID BOOLEAN EXPRESSION

```
shrridharshan@SD:~$ nano bool.l
shrridharshan@SD:~$ nano bool.y
shrridharshan@SD:~$ yacc -d bool.y
shrridharshan@SD:~$ lex bool.l
shrridharshan@SD:~$ gcc lex.yy.c y.tab.c -o bool -ll
shrridharshan@SD:~$ ./bool
Enter Boolean expression:
a AND OR b
TOKEN: ID (a)
TOKEN: AND
TOKEN: OR
Syntax Error in Boolean Expression
```

### LEXICAL ERROR INPUT

```
shrridharshan@SD:~$ nano bool.l
shrridharshan@SD:~$ nano bool.y
shrridharshan@SD:~$ yacc -d bool.y
shrridharshan@SD:~$ lex bool.l
shrridharshan@SD:~$ gcc lex.yy.c y.tab.c -o bool -ll
shrridharshan@SD:~$ ./bool
Enter Boolean expression:
a AND @ b
TOKEN: ID (a)
TOKEN: AND
Lexical Error: @
Syntax Error in Boolean Expression
```

## **RESULT**

The syntax analyzers were successfully built using recursive descent parsing and Lex Yacc techniques. The programs correctly checked arithmetic expressions, assignment statements, and Boolean expressions by applying grammar rules. Valid inputs produced appropriate success messages. Invalid inputs generated clear syntax and lexical error messages, showing effective syntax analysis.